

Name: Shaik Mohammed Abrar

Reg no: 192472185

Course Name: Design and Analysis of Algorithm

Course Code: CSA0652

Assessment: CO5-Assessment Tool 2

Report 2

Q55: Combination Generation with Sum Constraint Using Backtracking

1. Aim

The aim of this experiment is to generate all possible combinations of size **k** from a given set of numbers such that the sum of the selected elements equals a specified target value. The problem is solved using the **Backtracking technique** with pruning to improve efficiency by eliminating invalid combinations early.

2. Problem Description

Combination generation is a common combinatorial problem where a subset of elements is selected from a larger set. In this problem, only combinations that satisfy the following conditions are considered valid:

1. Exactly **k elements** must be selected.
2. The sum of selected elements must equal the target value.
3. Each element can be selected only once.
4. All valid combinations must be generated.

Since the number of possible combinations grows rapidly with the size of the set, Backtracking is used to systematically explore feasible solutions while pruning unnecessary branches.

3. Objectives

1. Generate all combinations of size **k**.
2. Ensure the sum of selected elements equals the target value.
3. Apply Backtracking for systematic exploration.
4. Use pruning techniques to reduce computation.
5. Display all valid combinations satisfying the constraints.

4. Algorithm

Step 1: Input : Read:

1. Set of numbers
2. Combination size **k**
3. Target sum

Step 2: Initialize

Create an empty list to store the current combination.

Step 3: Recursive Selection

Select elements one by one starting from the current position.

Step 4: Check Constraints

For every recursive call:

1. Add the current element to the combination.
2. Update the current sum.
- 3.

Step 5: Pruning

If the current sum exceeds the target value:

1. Stop exploring that branch.
2. Return to the previous state.

Step 6: Valid Combination Check

If:

1. Combination size = k
2. Current sum = target

Then:

3. Store or print the combination.

Step 7: Backtracking

Remove the last selected element and continue exploring alternative choices.

Step 8: Output

Display all valid combinations.

5. Backtracking Strategy

The algorithm follows a depth-first search approach:

1. Select an element.
2. Recursively explore further selections.
3. If constraints are violated, backtrack.
4. Continue until all valid combinations are discovered.

6. Pruning Techniques Used

a. 6.1 Sum-Based Pruning

If the current sum exceeds the target value:

Current Sum > Target

The branch is terminated immediately.

b. 6.2 Combination Size Pruning

If the combination already contains **k elements**:

Length(Current Combination) = k

The algorithm checks validity and stops further expansion.

c. 6.3 Index-Based Selection

Elements are selected only from subsequent positions to avoid duplicate combinations.

These pruning methods significantly reduce unnecessary recursive calls.

7. Example

Input

Set = {1, 2, 3, 4, 5}

k = 3

Target = 9

Execution

Possible combinations of size 3:

{1,2,3} = 6

{1,2,4} = 7

{1,2,5} = 8

{1,3,4} = 8

{1,3,5} = 9 ✓

{1,4,5} = 10

{2,3,4} = 9 ✓

{2,3,5} = 10

$\{2,4,5\} = 11$

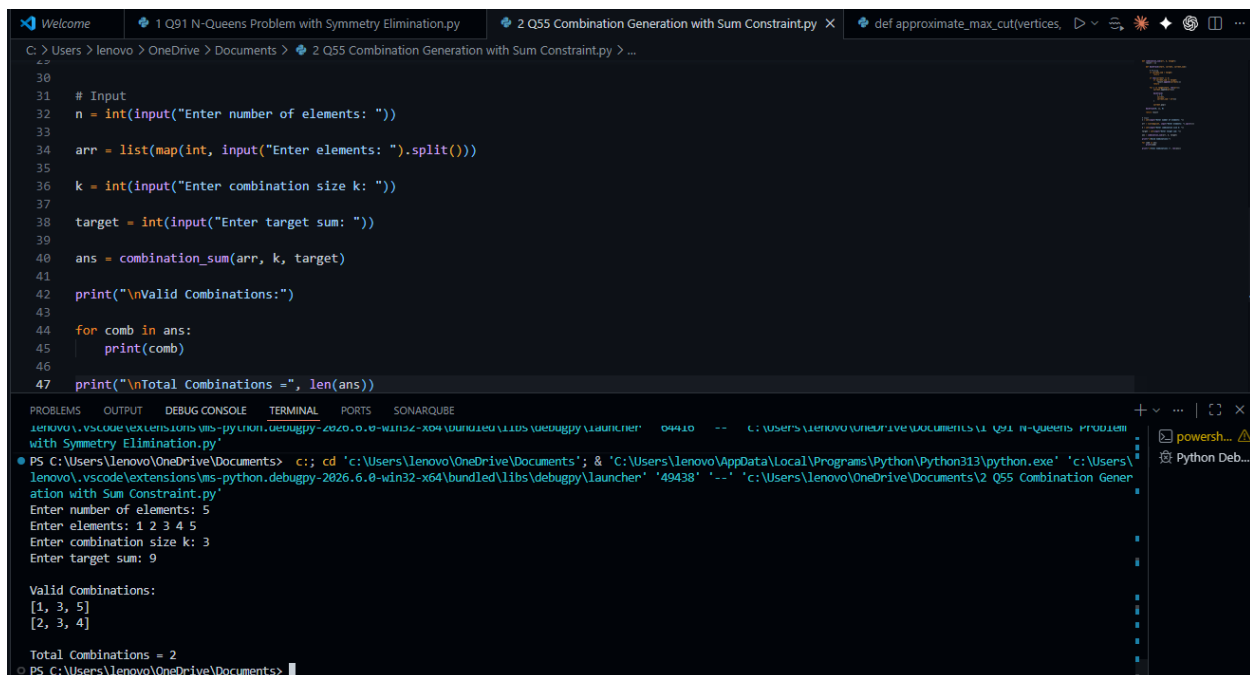
$\{3,4,5\} = 12$

Output

[1, 3, 5]

[2, 3, 4]

Total Combinations = 2



The screenshot shows a VS Code editor with a Python file named '2 Q55 Combination Generation with Sum Constraint.py'. The code defines a function 'approximate_max_cut(vertices)' and a main script that takes user input for the number of elements, the combination size k, and a target sum. It then prints valid combinations and the total count. The terminal output shows the script being executed with inputs: 5 elements, k=3, target sum=9. The output lists two valid combinations: [1, 3, 5] and [2, 3, 4], and states that the total number of combinations is 2.

```
30
31 # Input
32 n = int(input("Enter number of elements: "))
33
34 arr = list(map(int, input("Enter elements: ").split()))
35
36 k = int(input("Enter combination size k: "))
37
38 target = int(input("Enter target sum: "))
39
40 ans = combination_sum(arr, k, target)
41
42 print("\nValid Combinations:")
43
44 for comb in ans:
45     print(comb)
46
47 print("\nTotal Combinations =", len(ans))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SONARQUBE
lenovo\\.vscode\\extensions\\ms-python\\.debugpy-2020.0.0-win32-x64\\bundled\\libs\\debugpy\\launcher
with Symmetry Elimination.py'
PS C:\\Users\\lenovo\\OneDrive\\Documents> c;; cd 'c:\\Users\\lenovo\\OneDrive\\Documents'; & 'C:\\Users\\lenovo\\AppData\\Local\\Programs\\Python\\Python313\\python.exe' 'c:\\Users\\lenovo\\.vscode\\extensions\\ms-python\\.debugpy-2020.0.0-win32-x64\\bundled\\libs\\debugpy\\launcher' '49438' '-.' 'c:\\Users\\lenovo\\OneDrive\\Documents\\2 Q55 Combination Gener
Enter number of elements: 5
Enter elements: 1 2 3 4 5
Enter combination size k: 3
Enter target sum: 9

Valid Combinations:
[1, 3, 5]
[2, 3, 4]

Total Combinations = 2
PS C:\\Users\\lenovo\\OneDrive\\Documents>

8. Correctness of the Algorithm

The algorithm is correct because:

1. Every possible combination is explored systematically.
2. No combination is repeated.
3. The recursive procedure considers all valid element selections.
4. Pruning removes only impossible branches.
5. A combination is accepted only when:
 - o Exactly k elements are selected.

- Their sum equals the target value.

Thus, all valid combinations are generated and no valid solution is missed.

9. Complexity Analysis

Time Complexity

In the worst case, all combinations of size k must be explored:

$$O(C(n, k))$$

where:

- n = number of elements
- k = combination size

The binomial coefficient represents the total number of possible combinations.

Space Complexity

$$O(k)$$

Space is required for:

- a. Recursive call stack
- b. Current combination storage

10. Advantages

1. Efficiently generates valid combinations.
2. Avoids duplicate combinations.
3. Uses pruning to reduce unnecessary computations.
4. Easy to implement and understand.
5. Suitable for combinatorial search problems.

11. Applications

1. Resource Allocation Problems
2. Team Formation and Selection
3. Subset Sum Problems
4. Scheduling Systems
5. Financial Portfolio Selection
6. Data Mining and Pattern Discovery
7. Artificial Intelligence Search Problems
8. Constraint Satisfaction Problems (CSP)

12. Result

The Combination Generation with Sum Constraint problem was successfully solved using the Backtracking technique. The algorithm efficiently generated all combinations of size **k** whose sum matched the target value. Pruning strategies eliminated infeasible branches early, reducing computational overhead and improving performance. The experiment demonstrates the effectiveness of Backtracking for solving constrained combinatorial optimization problems.